Duo · Documentation technique

Duo est une application web mobile d'abonnement « 1 acheté = 1 offert ». Le membre s'abonne pour 12 mois, trouve une offre chez un commerçant partenaire et la fait valider en caisse avec le code à 4 chiffres du commerçant ou un QR code. Le commerçant présente son commerce, crée ses offres, choisit leurs jours et suit leur utilisation. Lien de démo : **https://projet-duo-delta.vercel.app** (branche `main`).

1. Architecture

Couche	Choix
Interface	React 19, Vite, JavaScript ; Tailwind CSS v4 (thème « crépuscule ») ; React Router (routes dans `src/App.jsx`) ; Three.js chargé à la demande pour le carrousel des menus
Données et sécurité	Supabase : authentification par e-mail, PostgreSQL, sécurité au niveau des lignes (RLS), fonctions SQL. Client unique : `src/lib/supabase.js`
Hébergement	Vercel, relié au dépôt GitHub : chaque fusion sur `main` met à jour le lien de démo
Qualité	Vitest (`npm test`), oxlint (`npm run lint`), build Vite avant chaque PR

Le navigateur parle directement à Supabase avec la clé publique : **toute la sécurité est dans la base** (RLS et fonctions), jamais dans le code de l'interface. Les écrans sont pensés pour 375 px de large. Les bibliothèques QR (`qrcode`, `qr-scanner`) ne se chargent que sur les écrans qui en ont besoin.

2. Modèle de données (`supabase/schema.sql`)

Table	Contenu
`profiles`	Un profil par compte : prénom, rôle `membre` ou `commercant` (créé à l'inscription)
`subscriptions`	Abonnement du membre : statut, début, fin (une ligne par membre)
`merchants`	Fiche du commerce : nom, catégorie (RG11), adresse, ville, description, photo
`merchant_secrets`	Code à 4 chiffres du commerçant, **haché** (bcrypt)
`offers`	Offres : titre, conditions, jours valables (ISO, lundi = 1), valeur offerte, `max_par_membre`, active
`redemptions`	Utilisations validées : offre, membre, date, numéro de validation, économie figée
`redemption_tokens`	QR codes de validation (bonus US14) : code court, expiration, utilisation

3. Sécurité

- **RLS activée sur toutes les tables.** Un membre ne lit que son profil, son abonnement et ses utilisations ; un commerçant ne gère que son commerce et ses offres. Un visiteur non connecté ne lit que les offres actives et les commerces (catalogue public).
- **Écritures sensibles uniquement par des fonctions** `security definer`, qui font leurs propres contrôles : `activer_abonnement`, `definir_code`, `valider_offre`, `creer_jeton_qr`, `valider_qr`. Aucun droit d'écriture direct sur `subscriptions`, `redemptions`, `merchant_secrets` ni `redemption_tokens` : un membre ne peut ni s'offrir un abonnement, ni créer une utilisation, ni lire un code, même en interrogeant l'API directement.
- **Code commerçant** : stocké haché, jamais relu ni réaffiché ; `code_defini` dit seulement s'il existe.
- **Secrets** : clés dans `.env` (local, ignoré par Git) et dans Vercel ; modèle : `.env.example`.

4. Règles de gestion et où elles sont appliquées

Règle	Application
RG01 accueil selon le rôle	`RouteProtegee`, `accueilDuRole` (front) ; rôle stocké dans `profiles`
RG02 abonnement de 12 mois	`activer_abonnement` : fin calculée par la base, un seul abonnement actif
RG03 visiteur invité à créer un compte	Page d'offre : « Utiliser » ouvre l'invitation
RG05 jour de validité	Jour courant **à l'heure de Paris** : `extract(isodow from now() at time zone 'Europe/Paris')`
RG06 limite par membre	`offers.max_par_membre` (1 à 10, choisi par le commerçant) ; `utilisations_restantes`
RG07 valeur offerte	Obligatoire, positive (contrainte sur `offers`)
RG08 / RG09 code à 4 chiffres	`definir_code` : 4 chiffres, haché, jamais réaffiché
RG10 offre utilisée	Pas supprimée, désactivée ; ses utilisations restent comptées
RG11 catégories	Liste fermée (contrainte SQL et `src/lib/categories.js`)
RG12 économies	`mes_economies` : total depuis le début de l'abonnement en cours, économie figée à la validation
RG13 QR code	Une offre, 5 minutes, une seule fois (`redemption_tokens`)

5. Validation en caisse

`valider_offre(offre, pin)` est le seul chemin d'écriture des utilisations. Elle fait **quatre contrôles**, dans l'ordre, et n'enregistre rien si l'un échoue :

1. abonnement actif du membre (`abonnement_inactif`) ;
2. offre active et valable aujourd'hui (`offre_non_valable`) ;
3. limite du membre pour l'offre (`limite_atteinte`) ;
4. code du commerçant (`code_incorrect`).

Si tout passe, elle enregistre l'utilisation et renvoie le **numéro de validation**, l'heure et l'économie, affichés en grand. Un verrou empêche deux validations simultanées de dépasser la limite. **QR code (US14)** : `creer_jeton_qr` produit un jeton (QR + code court de 6 caractères) ; `valider_qr`, appelée par le commerçant connecté, refuse un jeton inconnu, expiré, déjà utilisé ou d'un autre commerce, puis applique **les mêmes contrôles 1 à 3** (fonctions internes `controler_utilisation` et `enregistrer_utilisation`, non appelables par l'API). L'écran du membre suit l'état du jeton (`etat_jeton`) et affiche seul « Offre validée ».

6. Installer et lancer

1. `npm install`, puis `cp .env.example .env` et renseigner `VITE_SUPABASE_URL` et `VITE_SUPABASE_ANON_KEY` (clé publique du projet Supabase).
2. Dans l'éditeur SQL de Supabase, jouer **`supabase/schema.sql`** (rejouable), puis **`supabase/seed-demo.sql`** pour les données de démo (il affiche les identifiants à utiliser). Copier les scripts depuis l'éditeur de code et lancer sans sélection.
3. Côté Supabase : fournisseur e-mail activé, confirmation d'e-mail désactivée pour la démo.
4. `npm run dev` (http://localhost:5173), `npm run build`, `npm run lint`, `npm test`.
5. Vercel : déclarer les deux variables pour Production et Preview, puis redéployer.

7. Organisation du travail

Backlog dans Jira (clé **SCRUM**). Une branche par story, créée depuis le ticket et nommée avec sa clé ; une PR par story vers `preprod`, relue par l'autre développeur, puis `preprod` vers `main` pour la démo. Une story est terminée quand ses critères sont vérifiés, sa PR fusionnée et la fonctionnalité visible sur le lien de démo. Toute modification de base passe par `schema.sql`.